

SbD-ToE MCP: Coverage-Preserving, Ontology-Grounded Retrieval for Security-by-Design Code Generation

Pedro Farinha

Shiftleft – Secure Software Engineering, Lda.

Lisbon, Portugal

ORCID: 0009-0001-0569-9020

pedro.farinha@shiftleft.pt

Abstract—LLM-based coding assistants are increasingly used in software development, but the security of their output is hard to audit: prompts are informal and rarely retained in the engineering record, plain retrieval-augmented generation (RAG) over vector similarity can return plausible guidance without preserving the coverage of the source corpus or proving that all applicable requirements were considered, and generated code can silently omit prescribed controls or evidence. We demonstrate SbD-ToE MCP, an installable Model Context Protocol server that gives any compatible IDE assistant coverage-preserving, ontology-grounded retrieval over a fifteen-chapter security-by-design practitioner manual. The manual is a practitioner-authored prose corpus; its current version is demonstrated to cover, at minimum, the thirty-one external sources — frameworks, standards, and regulations — normalised against the typed ontology by the programme’s coverage-preserving compilation method. The manual content is bound to AppSec Core v1, a typed ontology of 259 instances across ten domain slices. The retrieval pipeline is deterministic: no generative model participates in matching or context assembly, so what reaches the agent is auditable byte-for-byte. The tool ships pre-bundled via npm, runs offline, and integrates with Claude, GitHub Copilot, Cursor, Windsurf, Zed, and other MCP-compatible clients. The contribution is not better prompting; it is a different grounding regime — structured retrieval, typed security concepts, provenance on every result, and expected evidence. What we claim by construction is coverage-preserving, auditable context assembly; selection precision/recall and a formal completeness invariant at retrieval time are out of scope here and are the subject of the pre-registered companion study [6]. A four-minute screencast walks through one task taken from MVP to compliant production: a grounded refactoring plan, CI/CD hardening with SBOM generation, SHA-256 checksums, and evidence bundling, DORA-oriented assessment whose findings can feed concrete downstream artefacts (backlog items, Jira tickets, remediation tracking, compliance road maps), and a regulatory-overlay-driven remediation in an existing logger.

I. INTRODUCTION

LLM-based coding assistants are increasingly used in software development, but the security of their output is hard to audit. Prompts are informal and rarely retained in the engineering record. Plain retrieval-augmented generation (RAG) over vector similarity can return plausible guidance without **preserving the coverage of the source corpus** or proving that **all applicable** requirements were considered. Generated

code can silently omit prescribed controls or evidence. Broad asks such as “make this secure” are too large for accountable automation.

We demonstrate **SbD-ToE MCP** (@shiftleftpt/sbd-toe-mcp), an installable server speaking the Model Context Protocol [9] (MCP) that any compatible IDE assistant can call to obtain **coverage-preserving, ontology-grounded retrieval** over a fifteen-chapter security-by-design (SbD) practitioner manual [1]. The manual is a practitioner-authored prose corpus whose current version is demonstrated to cover, at minimum, thirty-one external sources — frameworks, standards, and regulations — normalised against the typed ontology by the programme’s coverage-preserving compilation method [3]; the demonstration that the present manual covers those sources is reported in [4]. The retrieval surface is bound to AppSec Core v1, a typed ontology of 259 typed instances across ten domain slices [2]. The server ships pre-bundled, runs offline, requires no API keys, and integrates with Claude, GitHub Copilot, Cursor, Windsurf, Zed, and other MCP-compatible clients via `stdio`.

This is the tool a developer actually installs and uses. A companion pre-registered study [6] uses the stricter experimental apparatus specified in [5] for confirmatory evaluation; this paper does not duplicate that material.

Scope of the claim. What we claim *by construction* is coverage-preserving, auditable context assembly: the retrieval-time substrate carries a coverage account from compilation, every result carries provenance, and the scope gate refuses to fabricate identifiers. Selection precision/recall and a formal completeness invariant at retrieval time are out of scope for this paper and are the subject of the pre-registered companion study [6].

Contributions. Two research contributions, plus a continuous-task demonstration documented in Section~IV. (i) An end-to-end ontology-grounded retrieval tool for the application-security (AppSec) domain, usable in daily practitioner workflows. (ii) A deterministic retrieval pipeline whose output is auditable byte-for-byte. The demonstration walks one task from MVP to compliant production across

grounded refactoring planning, CI/CD hardening (SCA gate, SBOM generation, SHA-256 checksums, evidence bundling), DORA-oriented assessment whose findings can feed concrete downstream artefacts (backlog items, Jira tickets, remediation tracking, compliance road maps), and a regulatory-overlay-driven remediation in an existing file.

Key claim. *The contribution is not better prompting; it is a different grounding regime — structured retrieval, typed security concepts, provenance on every result, and expected evidence.*

Paper organisation. Section~II describes what the tool consumes from the upstream programme. Section~III specifies its architecture. Section~IV walks through the demonstration. Section~V states availability. Sections~VI–VII close with limitations, roadmap, and AI disclosure.

II. WHAT THE TOOL CONSUMES

The tool bundles four artefacts from the upstream AppSec Core / SbD-ToE research programme [1]:

- **The practitioner manual** — fifteen chapters (numbered 00–14) of practitioner-authored prose covering the SbD domain. The manual is not the output of an automated pipeline; it is human-written. What the upstream programme demonstrates is that the current version of the manual provides prose coverage for, at minimum, the thirty-one external sources normalised by the programme’s coverage-preserving compilation method [3]; the demonstration of that coverage is reported in [4]. *Coverage-preserving* in this work names a specific operational property of the compilation method: every source item admitted into the compilation boundary is explicitly routed (Core-mapped, Manual-only, or Out-of-AppSec) and retained in traceability records; nothing is silently dropped at compile time. The relevance: when a developer asks the tool a question, the answer cites back to manual content whose source-coverage relationship has been audited upstream.
- **AppSec Core v1** [2] — a typed ontology of 259 instances populated across ten domain slices and five entity classes (ControlObjective, Practice, Mechanism, Artifact, and the declarative EvidencePattern class). The slices partition the application-security domain; the classes are the typed entity vocabulary used within each slice. The ontology is not used for generation; it constrains retrieval semantics and stabilises cross-framework concept identity. It is the hub through which heterogeneous frameworks (SSDF, ASVS, SLSA, CIS Controls, CAPEC, OWASP SAMM, OWASP DSOMM), regulations (DORA, NIS2, CRA, GDPR), and AI/ML-specific guidance converge to common names. The relevance: identifiers in the tool’s responses point to entities in this ontology, not free-form text.
- **A regulatory overlay** that connects the ontology to specific regulatory obligations across major EU regimes (DORA, NIS2, CRA, GDPR). The relevance: when a developer asks for a DORA-oriented review — or, just as well, NIS2, CRA, or GDPR — the tool resolves

which ControlObjectives intersect with which regulatory articles, so the answer is anchored in the regulation rather than restated from the model’s prior knowledge of it.

- **A runtime knowledge graph** that links manual content to ontology entities. The relevance: this is what makes “find the parts of the manual relevant to *this* code change” a deterministic lookup rather than a similarity search.

The bundle is shipped with the tool and works offline.

Note on the cited programme companions. References [1]–[6] are deposits of the same research programme, archived at OSF in 2026 and at various stages of peer review at the time of writing. This paper relies on them as the programme’s own provenance chain rather than as independently established literature; the load-bearing claims this paper offloads to [3] (the compilation method) and [4] (the demonstration of substrate coverage) are themselves part of that chain.

A. Position against retrieval genres

A growing family of retrieval-augmented generation variants — ontology-grounded RAG (OG-RAG) [8] and the recent graph-RAG family typified by [7] — augments plain vector-similarity RAG by adding a structured retrieval substrate. The substrate this tool retrieves over is built differently. It is the output of a deterministic, coverage-preserving compilation [3, 4] that routes every source item explicitly — Core-mapped, Manual-only, or Out-of-AppSec — and distinguishes genuine content gaps from traceability artefacts at compile time, not at query time. The differentiator we elevate here is the **pre-serving** discipline: nothing in the corpus is silently dropped, every routing decision is logged, and the agent retrieves over a substrate that has already passed coverage analysis. Stated as an invariant: this approach commits to an explicit substrate-accountability invariant established at compile time — every source item routed and retained in traceability records before retrieval exists. Plain vector-similarity RAG provides no comparable property over the source corpus; whether the cited structured-substrate variants commit to a comparable invariant depends on each method’s construction and is not surveyed here.

III. TOOL ARCHITECTURE

A. Distribution

The tool is published as the npm package `@shiftleftpt/sbd-toe-mcp` (Apache-2.0 code, CC-BY-SA-4.0 data). The reference invocation is one command:

```
npx -y @shiftleftpt/sbd-toe-mcp
```

No API keys, no cloud calls, no internet at runtime. Client configurations are provided for Claude Code, Claude Desktop, Cursor, Windsurf, Zed, and VS Code with GitHub Copilot. A self-contained offline bundle is also published as a GitHub release asset for air-gapped environments.

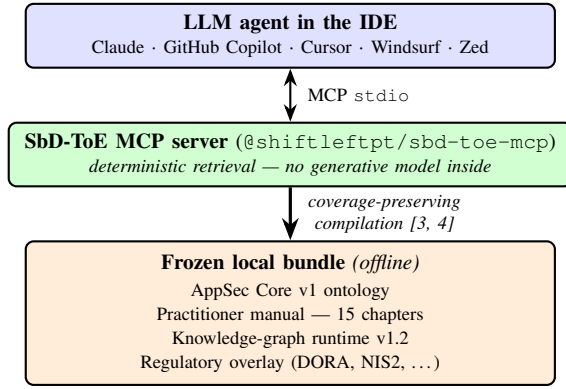


Fig. 1. An IDE-side LLM agent calls the SbD-ToE MCP server over MCP `stdio`. The server runs no generative model; it deterministically retrieves over a frozen, offline bundle of programme outputs. The bundle itself is produced upstream by a coverage-preserving compilation pipeline that routes every source item before retrieval time.

B. Architecture at a glance

Figure~1 sketches the pipeline. An IDE-side LLM agent calls the SbD-ToE MCP server over the standard `stdio` transport; the server reads a frozen, offline bundle of programme outputs and returns a deterministically-assembled, citation-backed context to the agent.

C. Tools and resources

The MCP interface exposes fifteen tools and six resources. The most consequential surfaces day-to-day are search and grounded Q&A over the manual (`search_sbd_toe_manual`, `answer_sbd_toe_manual`), context assembly for code-generation tasks with an explicit scope gate (`prepare_sbd_toe_codegen_context`, introduced in v0.9.0), entity resolution across runtime, ontology, and overlay (`resolve_entities`), review-scope mapping from changed file paths (`map_sbd_toe_review_scope`), and a debug surface exposing scores and selection rationale (`inspect_sbd_toe_retrieval`). The complete list, together with the six resources — including a compact chapter index injectable into a system prompt and an agent-facing operational guide — is documented in the project README. The scope gate is a first-class refusal surface: when scope inference would be unsafe, the tool returns an explicit clarification state (`needs_clarification`, `needs_decomposition`, or `unsupported_scope`) rather than fabricating ontology bindings to keep the agent moving.

D. What the tool guarantees, and what it doesn't

Three properties are worth stating directly.

Determinism. No generative model runs inside the tool. Matching, candidate selection, and context assembly are all deterministic. Inputs being equal, two invocations return bit-identical context. The only LLM in the loop is the one the developer is already running in their IDE, which consumes

what this tool delivers. Determinism is not a stylistic preference: non-deterministic retrieval undermines reproducibility of the security rationale and the evidence reconstruction the manual binds to, and auditability requires that two reviews of the same input return the same context.

Provenance on every result. Every item the tool returns carries citations — to a manual chapter, an ontology entity, or a regulatory overlay entry — that an agent can show to the user or attach to a pull request. The scope gate (`prepare_sbd_toe_codegen_context`) refuses to fabricate identifiers when context inference would be unsafe; it returns `unsupported_scope` or `needs_clarification` instead.

The substrate, and what we claim about it. The tool runs against a knowledge graph it does not build. How that KG was compiled, how source items were routed, and the evidence that the bundled outputs are encompassing of the AppSec corpus at their cycle-close state are reported in the companion programme papers — the method in [3] and the cycle demonstration in [4]. Within this paper we treat the KG as a frozen input: the tool calls into it, returns citations from it, and inherits the coverage that the upstream cycle produced. We do not claim the KG is perfect; refining and pressure-testing it further is ongoing programme work and is not in the scope of this tool. Whether the tool, on top of that substrate, achieves a formal completeness invariant at retrieval time is the subject of a companion pre-registered study [6].

IV. DEMONSTRATION

The accompanying screencast (4~min 12~s, <https://youtu.be/igLQMMYZepQ>) walks through three workflows that a working developer recognises immediately.

Discovery and grounded refactoring planning. The screencast opens with the simplest possible question — “what is SbD-ToE?” — and shows the tool returning a citation-backed overview of the framework rather than the model’s prior knowledge of the same name. It then runs a concrete task: take an MVP web application and identify what would be needed to move it to a production-ready state from a security-by-design perspective. The tool returns a grounded, structured context — normalized security concepts, structured provenance, and a rationale scaffold — which the assistant uses to generate human-readable analysis and propose the next engineering actions. The plan lands against named requirements and explicit citations, not against the assistant’s prior beliefs about what secure code looks like.

Hardening the CI/CD pipeline. Still on the same MVP, the assistant modifies the existing CI/CD pipeline to make supply-chain security controls explicit. Four steps are added, each named by its manual-resolvable identifier: a software-composition-analysis gate (`SEC-L2-DST-SCA-GATE`, `npm audit --audit-level=high`), a CycloneDX SBOM generation step (`SEC-L2-DST-SBOM`), SHA-256 checksums over the lockfile and the SBOM (`SEC-L2-DST-CHECKSUM`), and an evidence-upload step that bundles them as a single `ci-security-evidence`

artefact (SEC-L2-DST-EVIDENCE). The SEC-L2-DST-* step names are not cosmetic: each one resolves back to the manual content it implements — the same citation discipline used elsewhere in the tool’s responses, applied here to YAML configuration. The exercise shows that grounded retrieval supports broader software-delivery workflows, not only source-code generation.

Assessment-oriented retrieval and concrete remediation. Switching mode, the assistant analyses the codebase against DORA-related security and operational expectations and returns contextualised findings tied to specific application and engineering artefacts — material that can feed downstream governance such as backlog generation, Jira integration, remediation tracking, or compliance-transformation road maps. The screencast then shows the corresponding remediation in action: the assistant proposes a concrete edit to the existing `logger.ts` so that logging behaviour aligns with the DORA-related operational and traceability requirements the manual surfaces through the regulatory overlay. The tool publishes the overlay as an external cross-check, not as a compliance verdict — that judgement remains with the human reviewer.

V. AVAILABILITY

- **Everything in one place (DOI-citable):** <https://osf.io/pgdr6/> (DOI 10.17605/OSF.IO/PGDR6) — the OSF deposit holds the bundle, the manifest, and the screencast together. Archival mirrors at figshare and B2SHARE.
- **Source code (Apache-2.0):** <https://github.com/Shiftleftpt/sbd-toe-mcp-poc>.
- **Installable runtime (v0.9.0):** `npm install @shiftleftpt/sbd-toe-mcp` (or `npm -y @shiftleftpt/sbd-toe-mcp` for zero-config).
- **Screencast:** <https://youtu.be/igLQMMYZepQ>.

VI. LIMITATIONS AND ROADMAP

The bundled manual and knowledge graph are at a fixed cycle-close state; programme cycles refresh the bundle and the tool is re-released alongside. The tool is scoped to AppSec engineering substance; broader governance content is routed elsewhere by the upstream programme. The system trades automation for auditability, traceability, and retrieval governance — a deliberate engineering decision rather than a scalability claim, and that bound is acknowledged here as a design choice, not a deficiency.

The natural next step is to evaluate whether this grounding regime actually improves completeness, auditability, and execution fidelity compared with ungrounded systems or plain retrieval-augmented generation. That evaluation is the subject of the pre-registered companion study [6]; aligning the released tool with the stricter contract its apparatus specification [5] requires — explicit token-budget instrumentation, the three-condition contrast (ungrounded baseline, plain RAG, contract-compliant MCP), and the validation protocol with pre-execution acceptance criteria — is the headline engineering work that bridges this tool and that evaluation. The tool

today is a useful product; growing it into the controlled-evaluation apparatus is a separate engineering programme.

VII. AI DISCLOSURE

Per IEEE policy on the use of generative AI in published works, two distinct uses of AI in this work are disclosed.

Manuscript preparation. Portions of this manuscript and the screencast script were drafted with Claude (Anthropic) and reviewed, edited, and verified by the author. The author retains full responsibility for the content, claims, and accuracy.

AI as the subject of the paper. The MCP server demonstrated here is itself a structured interface through which AI clients consume grounded context. This is the methodological substance of the work, not a content-generation event; the server returns deterministic retrieval results and does not invoke any generative model.

VIII. REFERENCES

- [1] P.~Farinha, “AppSec Core Research Programme: Prospectus,” OSF, 2026. <https://doi.org/10.17605/OSF.IO/7T849>
- [2] P.~Farinha, “AppSec Core v1: A Formalized Normalization Ontology for Application Security,” OSF, 2026. <https://doi.org/10.17605/OSF.IO/U9CRD>
- [3] P.~Farinha, “Pressure-Testing AppSec Core via a Design Science Cycle at 31-Source Scale,” OSF, 2026. <https://doi.org/10.17605/OSF.IO/3E8G5>
- [4] P.~Farinha, “Coverage-Preserving Compilation v2: 31-Source Pipeline with Joint Manual and Knowledge-Graph Production,” OSF, 2026. <https://doi.org/10.17605/OSF.IO/TXW8P>
- [5] P.~Farinha, “Engineering a Transparent MCP Instrument for Controlled Evaluation of Ontology-Grounded Code Generation,” OSF, 2026. <https://doi.org/10.17605/OSF.IO/KH8Y7>
- [6] P.~Farinha, “Empirical Evaluation of Ontology-Grounded Retrieval for LLM-Assisted Secure Code Generation (pre-registered design),” OSF Registration, 2026. <https://doi.org/10.17605/OSF.IO/H5AJE>
- [7] D.~Edge *et al.*, “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” arXiv:2404.16130, 2024. <https://arxiv.org/abs/2404.16130>
- [8] K.~Sharma, P.~Kumar, and Y.~Li, “OG-RAG: Ontology-Grounded Retrieval-Augmented Generation for Large Language Models,” in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Suzhou, China, 2025, pp.~32962–32981.
- [9] Anthropic, “Model Context Protocol Specification,” 2026. <https://modelcontextprotocol.io/specification>